

AI AGENT 学习路线 · 学习专题 002

LLM 基础 学习笔记（打印版）

11 个篇章 · 6 张矢量架构图

生成日期：2026-09-19 | 由 Markdown 笔记自动排版生成

目录

01	002 - LLM 基础 学习专题 导航	
02	00 - 背景篇：从语言模型到 LLM 时代	含架构图 ✓
03	002 fundamentals · 学习路线图	含架构图 ✓
04	01 - 问题篇：为什么工程师要懂 LLM 底层	
05	02 - 概念篇：核心概念精讲（12 张概念卡）	含架构图 ✓
06	03 - 原理篇：从 Token 到回答的完整旅程	含架构图 ✓
07	04 - 进阶篇：训练三部曲与对齐	含架构图 ✓
08	05 - 实战篇：用好推理 API（全代码）	含架构图 ✓
09	06 - 生产篇：成本、延迟、质量三角	含架构图 ✓
10	07 - 对比篇：2026 模型格局全景与选型	含架构图 ✓
11	08 - 自测篇：题库与面试题（主动回忆 + 间隔重复）	分级题库 ✓

共 11 个篇章。

002 - LLM 基础 学习专题

这是 AI Agent 学习路线的地基层。**Agent 的一切行为都建立在 LLM 的机制之上**——不懂 token 和注意力，就永远只能“调 API 的 Черный 盒”；懂了底层，你才能读懂官方文档里的每个参数、诊断线上怪象、在模型格局剧变时不慌。

📖 本专题使用的学习方法（与 001/008 一致）

方法	在笔记中的体现	你该怎么做
费曼技巧	每个概念先用【一句话人话】+【生活类比】，再进技术细节	读完后把"Transformer 在干什么"讲给同事听，卡壳处回读
黄金圈法则	01 先讲"为什么工程师要懂底层"，再讲"是什么"，最后"怎么用"	不要跳过 Why
康奈尔笔记 + 主动回忆	每章开头【本节问题】，结尾【本节自测】	读前扫问题，读后合上笔记作答
间隔重复	08-自测篇提供 D1/D3/D7/D21 计划	按计划重做错题

📖 专题目录

002-llm基础/	
├─ README.md	← 你在这里
├─ 00-背景篇-从语言模型到LLM时代.md	← 补充知识：NNLM→RNN→Transformer→GPT 全前史
└─ fundamentals/	← 主线：从 token 到 API 的完整链路
├─ README.md	学习路线图 + 概念地图
├─ 01-问题篇-为什么工程师要懂LLM底层.md	Why：不懂底层的四类翻车现场
├─ 02-概念篇-核心概念精讲.md	What：12 张概念卡（费曼式）
├─ 03-原理篇-从token到回答的完整旅程.md	How：一次前向计算的图解 + 伪代码
├─ 04-进阶篇-训练三部曲与对齐.md	预训练/SFT/RLHF/R1式RL + 缩放定律
├─ 05-实战篇-用好推理API.md	全代码：调用/流式/参数/本地部署
├─ 06-生产篇-成本延迟质量三角.md	计费结构 + 优化抓手 + 模型退役迁移
├─ 07-对比篇-2026模型格局全景.md	闭源三强+开源四杰 + 选型决策树
└─ 08-自测篇-题库与面试题.md	L1-L4 + 间隔重复计划

🗺️ 学习路径建议

目标	路径	预计投入
快速了解（能对话）	README → 01 → 02 → 07 扫一眼	1~2 小时
系统掌握（能用好 API）	全部顺序读 + 跑通 05 的代码 + 08 的 L1/L2	1~2 天
深入精通（能选型+排障）	精读 03/04 → 06 成本建模实操 → 07 亲测对比 → 08 全过	3~5 天

∞ 与其他目录的关系

- 本专题是 001-prompt工程 的底层解释 (为什么 prompt 有效 → attention 机制)
- 03 篇的 KV cache / 上下文窗口是 008-记忆与上下文工程 的前提
- 07 篇的模型格局是 009-agent框架实战 选型的输入
- 学完可进 003-agent原理与架构 看"模型之外"的循环层

00 - 背景篇：从语言模型到 LLM 时代

【本节问题】① "语言模型"到底在建模什么？② 从 RNN 到 Transformer 发生了什么质变？③ BERT 和 GPT 的路线之争为什么 GPT 赢了？④ 从 2017 到 2026，哪些关键节点塑造了今天的格局？



1. 为什么要学前史？ (Why 补充课)

一句话人话：LLM 不是凭空出现的魔法，是一条 40 年的进化链。知道每个环节解决了什么问题，你才能理解今天每个"习以为常"的设计 (token、注意力、对话格式) 为什么长这样。

生活类比：用 Vue 的人不需要读过源码，但要懂"响应式"思想——不懂的人永远写不出正确的心智模型。LLM 同理，你要懂的核心思想是：**预测下一个 token**。

2. 语言模型在建模什么 (一个公式讲完)

语言模型的本质任务：**给定前文，计算下一个词的概率分布**。

$P(\text{下一个词} \mid \text{前文})$ — "点价数量是 ____" → "500"(概率0.62) "三"(0.05) "很"(0.03) ...

把这句话学会，奇迹就会涌现：语法（接得通顺）、事实（接得正确）、推理（多步接龙=思考）。所有惊人的能力，都是"预测下一个词"这件事练到极致后的副产品——这是理解 LLM 最重要的一个认知。

3. 四代技术演进（谁解决了谁的死穴）

代际	代表	怎么建概率	死穴
统计语言模型 (1990s)	N-gram	数前 2~3 个词的共现频率	只看近处，上下文窗口 ≈3 个词
神经网络语言模型 (2003-2017)	word2vec、RNN/LSTM	词变向量，RNN 逐词读、传隐藏状态	长序列信息衰减（记不住开头）；串行计算慢
Transformer (2017)	《Attention Is All You Need》	自注意力：任意两个位置直连 ，并行计算	（没有致命死穴，但有算力开销 n^2 ）
LLM 时代 (2018-今)	GPT/BERT→GPT-5/Claude/Gemini/DeepSeek	Transformer + 海量数据 + 疯狂堆规模	对齐、幻觉、成本（当前战场）

RNN → Transformer 的质变（费曼版）：RNN 像传话游戏——第 1 个人的话传到第 100 个人嘴里早变了样；Transformer 像**开会**——每个人（token）都能直接看到并"注视"在场的所有人，谁重要看谁，信息不衰减，而且所有人同时发言（GPU 并行，训练速度起飞）。

4. 2018 年的分岔口：BERT 路线 vs GPT 路线

	BERT (2018-10)	GPT (2018-06 起)
结构	Transformer Encoder (双向)	Transformer Decoder (单向)
训练目标	完形填空（遮住中间猜）	接龙（看前文猜下一个）
天赋	理解（分类、匹配、抽取）	生成（续写、对话）
使用方式	微调后用	Zero/few-shot 直接提示 （GPT-3 2020 证明）
结局	统治 NLP 竞赛 3 年，随微调范式一起退场	长成今天的 LLM 全部主流

胜负手：GPT 路线"一个模型 + 自然语言指挥"的通用性碾压了"一个任务微调一个模型"的效率。2017 论文里的完整 Encoder-Decoder（翻译架构），今天只剩 Decoder 独大——但 Encoder 的技术遗产（embedding、注意力）活在 RAG 检索模型里（005 专题）。

5. LLM 时代大年表 (2017-2026 关键节点)

2017	Transformer 论文 (Google)	— 架构地基
2018	GPT-1 / BERT	— 路线分岔
2020	GPT-3: 175B, 发现 ICL	— "提示"成为方法论 (→001 专题)
2022	InstructGPT (RLHF) + CoT 论文	— 会听话 + 会推理
2022.11	ChatGPT 发布 (5 天百万用户)	— 走入大众
2023	Llama 开源 → 开源社区爆发	— 权力下放
2023	GPT-4 / Claude 2: 100k 上下文	— 长上下文竞赛开始
2024	o1 发布: 推理时思考 (test-time compute)	— 第二缩放定律: 思考越多越聪明
2024.12	DeepSeek-V3 (MoE 671B/37B)	— 开源追平闭源的开端
2025.01	DeepSeek-R1 开源推理模型	— R1式RL训练范式公开
2025	MoE 全面主流: 1M 上下文逐步标配	— 效率竞赛
2026	GPT-6 Astra / Claude Opus 5 / Gemini 3.1	1M 上下文成旗舰标配; 思考 token 计费成常态; Agent 专用优化

6. 2026 年格局的三个结构性事实 (07 篇展开)

- MoE 成为绝对主流:** 从 DeepSeek 到 GPT-5.6 系, "总参数巨大、每 token 只激活一小部分"是同时做大容量和低成本的唯一解。看懂"总参/激活参" (如 1.6T/49B) 是读懂 2026 模型新闻的基本功。
- 上下文战争进入"质量"阶段:** 1M token 是旗舰标配 (table stakes), 比拼的是**满窗口的召回质量** (context rot 的对抗) 与长上下文计费策略。
- 思考 token 成为产品维度:** 推理模型把"算力换智能"做成可调参数 (reasoning_effort/effort/thinking budget), 并按输出计费——它既是能力来源也是成本大头。

【本节自测】

- 用一个公式说清语言模型在建模什么? 为什么说"惊人能力是副产品"?
- RNN 和 Transformer 的核心区别, 用"传话游戏 vs 开会"的类比展开讲。
- BERT 和 GPT 训练目标的差别? GPT 路线胜出的关键是什么?
- 2022 年的两篇关键工作 (InstructGPT/CoT) 分别解决了什么问题?
- o1 (2024) 开创的"第二缩放定律"和之前堆参数的缩放有什么本质不同?
- MoE 的"总参/激活参"是什么意思? 为什么 2026 年全是 MoE?
- 看图复述: 技术栈七层里, 哪几层是你写代码的、哪层是花钱的、哪层要懂原理?

002 fundamentals • 学习路线图

主线：从 token 到账单——沿一次 API 调用的生命周期，把 LLM 基础全部串起来。

概念地图（先看图建立全局）



章节地图

章节	黄金圈	回答的问题	配图
01 问题篇	Why	不懂底层会怎么翻车？	—
02 概念篇	What	12 个核心概念（费曼卡）	02 前向旅程
03 原理篇	How	一次前向计算内部发生什么？	02 + 03 KV缓存
04 进阶篇	How	模型是怎么被训练出来的？	04 训练三部曲
05 实战篇	How	API 怎么调用才专业？	06 API时序
06 生产篇	How	成本/延迟/质量怎么平衡？	06 成本结构
07 对比篇	扩展	2026 格局长什么样、怎么选？	05 模型版图
08 自测篇	闭环	都记住了吗？	—

建议阅读顺序

01 → 02 → 03（懂模型内部）→ 05 → 06（用好 API、管好钱）→ 04（懂训练，理解模型行为差异的根源）→ 07（看格局做选型）→ 08（自测闭环）。

时间紧：02 + 05 先行，其余按需。

01 - 问题篇：为什么工程师要懂 LLM 底层

【本节问题】① "会调 API 就行，原理是算法工程师的事"——这句话错在哪？② 不懂底层的人会在哪四个地方翻车？③ 懂到什么程度算够？

1. 黄金圈 Why：你是"用模型的人"，不是"造模型的人"——所以要懂的是"行为规律"

一句话人话：司机不需要会造发动机，但必须懂油门刹车油耗——否则一上高速就出事。LLM 工程师同理：不用会训模型，但必须懂 **token 经济、上下文窗口、采样随机性、思考成本** 这四个"驾驶仪表"。

2. 四类翻车现场（每一条都是真实高频事故）

翻车 ①：不懂 token —— 账单爆炸与中文截断

- 把一段 3 万字合同发给"128K 上下文"的模型，失败了——不知道**中文字≈token 的换算**和"上下文里还有 system+历史要一起算"
- 输出价格是输入的 5 倍，让模型"详细展开说说"——成本直接 $\times 10$
- 不知道 `max_tokens` 限制的是**输出**，以为是总长度

翻车 ②：不懂上下文窗口 —— "记忆"幻觉

- 以为模型记得上一轮会话（**每次调用都是失忆的**，是你自己把历史塞回去的）
- 以为窗口 1M = 可以无限塞文档（context rot：塞得越多，中间内容越容易被忽略）
- 多轮 Agent 循环里历史越滚越大，没人裁剪——延迟和成本双双爆炸

翻车 ③：不懂采样 —— 输出不可复现的困惑

- 同一个提示词两次结果不同就以为"有 bug"——其实 $temperature > 0$ 本来就掷骰子
- 对抽取任务用默认温度，格式漂移不知道为什么
- 反过来，以为 $temperature = 0$ 就一定正确（它只保证可复现）

翻车 ④：不懂推理模型 —— 成本黑洞与错误用法

- 给 R1/GPT-5 发一个"把日期格式转换一下"的简单任务——模型思考了 3000 token，成本 10 倍、延迟 10 倍
- 用提示词硬教推理模型"第一步...第二步..."——输出反而变差（001 篇讲过）
- 不知道思考 token 也按输出计费——月底账单里多出一大笔"看不见的消费"

3. 懂底层 → 读得懂官方文档的"潜台词"

官方文档黑话	底层知识	实际含义
"context window 1M"	02 卡5	输入+输出总和的上限，且长上下文可能加价
"reasoning tokens are billed as output"	02 卡10	思考过程也收输出价，Agent 场景成本大头
"cached input tokens"	02 卡12	前缀重复即可省钱，设计提示词要"稳定前缀在前"
"prefill is no longer supported"	03 篇	不能再偷改模型的开头，锁格式改用结构化输出
"batch API 50% off"	06 篇	异步任务换端口，直接半价

4. 懂到什么程度算够？（本笔记的度）

- ✓ 够用线：能解释 token/窗口/采样/缓存/思考成本 五个仪表盘；能看懂 07 篇的格局表并做选型
- ✓ 加分线：能画出一轮前向计算的流程（03 篇），能解释 KV cache 为什么省钱省时
- ✗ 不需要：手推反向传播、实现 attention 内核、复现 RLHF 训练——那是另一条职业路线

💡 费曼检验：向你的 Java 同事解释"为什么 LLM API 按 token 计费而不是按次"——因为每次请求的算力消耗与 token 数成正比（注意力 n^2 + 逐 token 生成），token 就是算力的计量单位。

【本节自测】

1. 四类翻车现场，各举一个你目前工作中可能踩的例子。
2. "模型记得上一轮对话吗？"——准确回答并说明记忆从哪来。
3. temperature=0 保证什么、不保证什么？
4. 思考 token 怎么计费？什么场景下它是成本大头？
5. 官方文档五句黑话，你能各翻译成一句"实际含义"吗？
6. 本笔记划定的"够用线"是什么？你现在的差距在哪？

02 - 概念篇：核心概念精讲（12 张概念卡）

【本节问题】token 怎么切？注意力在“注意”什么？MoE 的“激活参数”是什么？KV cache 为什么是性能钥匙？思考 token 是什么、怎么计费？



每张卡：【一句话人话】→【生活类比】→【技术要点】→【常见坑】

卡 1：Token 与 Tokenizer (BPE)

- **一句话人话**：token 是模型世界的“积木块”，tokenizer 是把文本切碎成积木的机器。
- **类比**：乐高——任何句子都能拆成标准件，模型只认标准件不认原句。
- **技术要点**：主流用字节级 BPE（高频组合成块：th + e → the）；英文 1 token≈0.75 词，**中文 1 字 ≈0.6~1 token**；每个模型词表不同（GPT 系 ~100k，DeepSeek/Qwen 对中文优化更大词表）。
- **坑**：按“字数”估算成本和长度必错；用 tokenizer 工具（各家 playground）实测才准；生僻 emoji/乱码可能被切成一堆 token，又贵又差。

卡 2：Embedding（嵌入向量）

- **一句话人话**：把 token 翻译成一串数字（向量），让“意思”变成几何距离。
- **类比**：给每个词发一张“语义身份证”，身份证号相近=意思相近。
- **技术要点**：token → 一个 768~4096 维向量；训练后语义空间自动成型（国王-男人+女人≈女王）；**独立的 embedding 模型**（bge、text-embedding 系列）把整段文本变成向量，用于 RAG 检索（005 专题地基）。

- **坑：**embedding 相似 $\neq$ 事实正确——向量检索到"语义相近"的内容就停，别拿它替代事实校验。

卡 3: Attention (自注意力)

- **一句话人话：**每个 token 环顾全句，决定"我该重点看谁"。
- **类比：**读合同的你，看到"违约金"三个字会自动重点扫"百分之""上限""日"这些关联词——注意力就是模型的"重点扫描"。
- **技术要点：**每个 token 生成 Query (我在找什么)、Key (我是什么)、Value (我携带的信息)；用 Q·K 算相关度 $\rightarrow$ 加权求和 V。多头=多个视角同时扫 (语法头/指代头/语义头)。**代价：n 个 token 两两看 n^2 次**——这是长上下文贵的根源。
- **坑：**以为"模型在读全文"——它是"加权平均地看"，中间内容权重低时 (lost in the middle) 就会漏。

卡 4: Decoder-only 与自回归生成

- **一句话人话：**整个大模型就是一个"接龙机"：看前文 $\rightarrow$ 吐一个 token $\rightarrow$ 拼回去 $\rightarrow$ 再吐。
- **类比：**成语接龙高手，永远只能接着你最后一个字说。
- **技术要点：**2026 所有主流 LLM 都是 decoder-only 自回归；"回答"是一次接龙几十~几千次的产物；**没有先想好再输出**——流式输出 (逐字蹦) 就是它的真实工作方式。
- **坑：**以为模型"组织好了语言再说"——其实每一个字都是即时预测；理解这点才能理解延迟结构和流式 API 的设计。

卡 5: 上下文窗口 (Context Window)

- **一句话人话：**模型的"工作台"，一次推理能摊开的最大 token 数。
- **类比：**办公桌尺寸——桌上的文件都看得见，桌外的一概不知。
- **技术要点：**2026 旗舰标配 1M (GPT-5.6 系 400K~1M、Claude Opus 5 / Sonnet 5 1M、Gemini 3.1 Pro 1M、DeepSeek V4 1M、Llama 4 Scout 理论 10M)；**输入+输出共享窗口**；扩长技术：RoPE 位置编码、YaRN 插值 (Qwen 原生 262K $\rightarrow$ 可扩 1M)、DeepSeek V4 压缩稀疏注意力。
- **坑：**① 窗口大 $\neq$ 质量好 (context rot: 注意力预算被摊薄，中间内容召回率降)；② 部分厂商超阈值加价 (Gemini >200K 翻倍、GPT-5.6 >272K $\times 2$)。

卡 6: 训练三部曲 (预训练 $\rightarrow$ SFT $\rightarrow$ 对齐)

- **一句话人话：**先通读互联网 (学知识)，再上岗位培训班 (学听话)，最后绩效价值观考核 (学规矩)。
- **类比：**学历 $\rightarrow$ 入职培训 $\rightarrow$ 绩效考核。
- **技术要点：**预训练 (万亿 token 下一词预测，99% 算力) $\rightarrow$ SFT (几万~百万条指令对) $\rightarrow$ 对齐 (RLHF/DPO/R1式RL)。详见 04 进阶篇。
- **坑：**混淆 Base 模型与 Chat 模型——拿 base 模型做对话会得到"续写你问题的文本"而不是回答。

卡 7：参数量、总参与激活参 (MoE)

- **一句话人话**：总参=图书馆藏书量，激活参=每次实际翻开的册数；MoE 就是"巨大图书馆 + 智能取书员"。
- **类比**：医院全科医院（稠密=每个病人惊动所有科室）vs 分诊制（MoE=按病情叫号，只开 2~3 个科室）。
- **技术要点**：2026 主流全是 MoE——DeepSeek V4-Pro **1.6T 总参/49B 激活**、V4-Flash 284B/13B、Llama 4 Maverick 400B/17B、Qwen3.6-35B-**A3B**（A=激活 3B）；路由器（router）为每个 token 挑专家；好处：容量大 + 单 token 成本低。
- **坑**：把总参当推理成本（决定显存的是总参，决定单 token 算力的是激活参）；"A3B"这类命名看不懂就读不懂 2026 模型新闻。

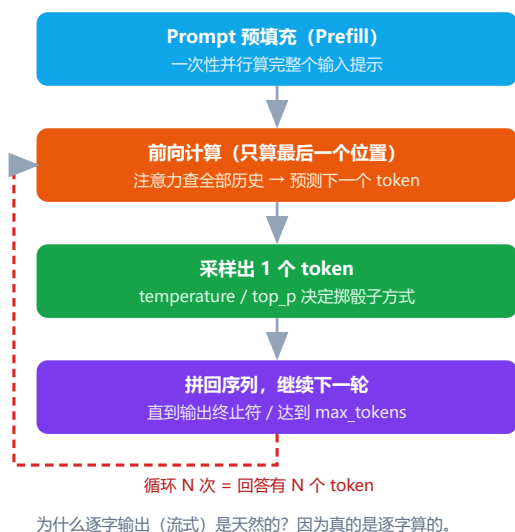
卡 8：KV Cache

- **一句话人话**：把历史 token 的"Key/Value 索引卡"存下来，生成新词时只算新词、查旧卡。
- **类比**：看书时给每页贴便利贴摘要——不用每次从头重读全书。
- **技术要点**：Prefill 阶段（读 prompt）一次性并行建卡 → Decode 阶段每步只算新 token 的 K/V 追加；推理速度的数量级来源。**代价：占显存，与上下文长度成正比**——这也是 prompt caching 能省钱的物理基础（前缀的 KV 直接复用）。
- **坑**：以为 API 服务器"算完就扔"——KV 在请求期间驻留显存，这就是并发有限价、长上下文贵的原因。

自回归生成循环与 KV Cache (推理提速的关键)

左：生成的循环结构 · 右：KV Cache 为什么能把每步计算量砍掉一个数量级

生成循环 (Decode 阶段)



第 100 步生成时，注意力要"查历史"

✗ 无 KV Cache: 每步全部重算

- 历史 99 个 token 的 K、V 全部重算一遍
- 第 200 步又要重算 199 个 重复劳动爆炸
- 总计算量随输出长度 $\approx$ 平方增长
- 相当于每次翻书都把整本书重新抄一遍再查

✓ 有 KV Cache: 历史算一次，存起来

- Prefill 时把每个 token 的 K、V 算好放进显存
- 之后每步只算"新 token"的 K、V，追加进缓存
- 每步计算量从 $O(\text{全部历史})$ 降到 $O(1 \text{ 个新 token})$
- 把书抄在便利贴上贴在手边，随用随取

代价：KV Cache 要占显存（与上下文长度成正比）——这就是"长上下文=贵"的物理原因，也是 Prompt Caching 能省钱的原因（前缀的 K/V 复用，不用重算）

卡 9: 采样参数 (temperature / top_p / top_k)

- **一句话人话**: 控制模型掷骰子的"歪度"——是照标准答案抄, 还是自由发挥。
- **类比**: temperature=0 的答题卡机器 vs 1.0 的即兴脱口秀演员。
- **技术要点**: 温度缩放 logits → softmax → 按概率抽; top_p (核采样: 只从累计概率 p 内的候选抽)、top_k (只从前 k 个抽); 抽取/分类/代码用 0~0.3, 创作用 0.7+。**2026 特情: Claude 4.6+ 禁用手动采样参数 (自适应取代), GPT-5 系用 reasoning_effort 管思考深度。**
- **坑**: temperature=0 ≠ 正确, 只是可复现; 参数解决不了提示词写得烂的问题。

卡 10: 思考 token 与推理模型

- **一句话人话**: 推理模型会在"正式回答前"先写一大段看不见的草稿 (思考链), 这段草稿也收输出费。
- **类比**: 请了个顾问, 报价包含"内部研究时间"——研究越久越贵也可能越准。
- **技术要点**: 训练出来的能力 (R1式 RL), 不是提示词技巧; 控制旋钮: GPT-5 reasoning_effort (low/medium/high)、Claude effort + 自适应思考、Qwen enable_thinking、DeepSeek V4-Flash/Pro 双型号; 思考内容通常给摘要 (reasoning_content / thinking 块)。**简单任务别用推理模型 (纯浪费)。**
- **坑**: 账单里思考 token 占输出 50%+ 很常见——Agent 长循环场景必须监控; 对推理模型写"step by step"是画蛇添足。

卡 11: 多模态 (视觉/音频输入)

- **一句话人话**: 给"文字接龙机"前面加一个翻译官——把图片/音频翻译成它认识的"token"。
- **类比**: 接入一位手语翻译, 聋哑人 (图片) 也能参会。
- **技术要点**: 视觉编码器 (ViT 系) 把图切块成向量, 与文本 token 同台进 attention (early fusion, Llama 4 的设计); 2026 旗舰全线支持图像输入 (GPT-5.6、Claude、Gemini、Qwen3.6 带 vision 编码器); DeepSeek V4 以文本见长, 多模态另有布局。
- **坑**: 图片 token 消耗远超想象 (一张图≈几百~几千 token); "看懂图"≠"看清小字" (表格截图识别仍有翻车率)。

卡 12: Prompt Caching 与计费结构

- **一句话人话**: 相同的前缀 (system+示例) 第二次调用直接用现成的 KV, 价格打 1~2 折。
- **类比**: 健身房年卡——第一次办卡贵, 之后每次进场几乎免费。
- **技术要点**: 命中条件=前缀**逐字节相同** (所以"稳定内容在前、可变内容在后"是成本设计原则); 各家折扣: 缓存命中约 0.1~0.2 倍计费 (Claude Fable 5.1 甚至 97.5% off); 思考 token 按输出价; batch 半价。
- **坑**: 把时间戳/随机 ID 放在 prompt 开头=缓存永久失效; 误以为缓存跨用户共享 (同租户同前缀才命中)。

【本节自测】

1. 为什么"按字数估成本"必错？中文 token 换算比例是多少？
2. 注意力的 n^2 代价和"长上下文贵"之间是什么关系？
3. 自回归生成和"流式输出"是什么关系？
4. 总参 1.6T/激活 49B 的模型，决定显存的是哪个数、决定单 token 算力的是哪个数？
5. KV cache 解决什么问题？它和 prompt caching 是一回事吗？（注意区分）
6. 推理模型的思考 token 怎么计费？什么任务不该用推理模型？
7. prompt caching 的命中条件是什么？你的提示词结构要怎么设计才能吃到缓存？
8. 12 张卡里哪三张直接解释了"LLM API 为什么这么贵"？贵在哪三个机制？

03 - 原理篇：从 Token 到回答的完整旅程

【本节问题】① 一次前向计算内部分几步？② Q/K/V 到底在算什么（不用数学）？③ 一句"500"是怎么被采样出来的？④ 推理速度瓶颈在 Prefill 还是 Decode？



② 嵌入 (Embedding) + 位置信息

- 查表：每个 token 映射为一个向量（如 4096 维）；再叠加**位置编码**（主流 RoPE 旋转位置编码）告诉模型“它在第几位”
- **为什么要位置**：注意力本身“天生无序”（集合视角），没有位置信息“猫追狗”和“狗追猫”就没区别

③ Transformer 块 ×N（核心车间）

每层块做两件事，结果残差连接累加：

a. 自注意力（互相看）

每个 token 派出三个分身：

- Q (Query, 我要找什么)
- K (Key, 我是什么标签)
- V (Value, 我携带的信息)

配对过程（费曼版）：

- 我用我的 Q 去和所有 token 的 K “对暗号” → 对上了（点积大）= 你重要
- 按重要度加权接收大家的 V → 得到“看完全句后的我”

- 多头：几十组 Q/K/V 并行（有的头盯语法、有的盯指代、有的盯数字）
- GQA（分组查询注意力）：KV 分身共用，显存大降——2026 标配

b. 前馈网络 FFN（深加工）

- 两层 MLP 把“看完全句的我”再深度变换，参数量占大头（专家知识存这里，MoE 的“专家”就是多套 FFN）

④⑤ 输出头 (Logits → 概率)

- 最后位置的向量 × 词表矩阵 → 每个词表 token 的原始得分 (logits)
- softmax + 温度： $\text{概率} \propto e^{(\text{logit}/T)}$ ——T 越小分布越尖（确定），T 越大越平（发散）

⑥ 采样

- temperature=0：直接取最大（贪心）
- top_p=0.9：按概率从高到低累计到 0.9 的候选池里抽
- 采到 500 → 拼回输入 → 回到 ①，直到终止符

3. 自回归循环与 KV Cache (性能的钥匙)

自回归生成循环 与 KV Cache (推理提速的关键)

左：生成的循环结构 · 右：KV Cache 为什么能把每步计算量砍掉一个数量级



```
# 伪代码: 把整个旅程压缩成 15 行
def generate(prompt, max_new_tokens):
    tokens = tokenize(prompt)
    kv_cache = []
    kv_cache = prefill(tokens, kv_cache)

    out = []
    for _ in range(max_new_tokens):
        logits = forward(last_token, kv_cache)
        prob = softmax(logits / temperature)
        new_tok = sample(prob, top_p)
        if new_tok == EOS: break
        out.append(new_tok)
        kv_cache.append(kv_of(new_tok))
    return detokenize(out)
```

Key/Value 索引卡盒
Prefill: 并行吃下全部输入, 建好所有卡
Decode: 逐 token 循环
只算新 token (旧 token 查卡即可)
温度调骰子
掷骰子
终止符
新卡入盒

两阶段延迟结构 (排障必备) :

阶段	在干什么	瓶颈	对应体感
Prefill	并行处理输入 prompt	算力 (大 prompt 一次算完)	首字延迟 TTFT——发 100k 文档后“卡住不动”就是在 prefill
Decode	逐 token 生成	显存带宽 (每步读全部 KV cache)	每秒输出 token 数 (吞吐)

4. 为什么这套设计能并行训练 (Transformer 的第二天赋)

- 训练时所有位置的目标都已知 → **一次算出全部位置的预测损失** (不需要像 RNN 那样串行)
- GPU 万卡并行的前提 → 才有"堆万亿 token 预训练"的可行性
- 这就是 2017 论文标题的野心: *Attention Is All You Need*——只注意力 (可并行), 不要循环 (慢)

5. 读懂 API 行为的四个推论 (原理的直接变现)

1. **长 prompt → 首 token 慢**: prefill 要算完全部输入 (也解释了 prompt caching 为什么有效——prefill 跳过)
2. **输出 token 贵且慢**: 逐 token 串行生成, 每个都要过全部 N 层
3. **多轮对话成本线性涨**: 历史拼回输入重新 prefill (除非缓存命中), 每轮都是全量再算
4. **max_tokens 截断是硬停止**: 循环到上限即停, 不管话说完没有——所以 `finish_reason: length` 不是异常是机制

【本节自测】

1. 凭记忆画六站流水线, 标注哪两站在 Transformer 块内循环。
2. Q/K/V 各自的角色? 用"对暗号"语言向同事解释注意力配对。
3. RoPE 解决什么问题? 没有位置信息的 attention 会犯什么错?
4. temperature 在 softmax 里的作用位置? 温度趋近 0 时分布怎么变?
5. TTFT 由哪个阶段决定? 长文档场景"卡住"在干什么?
6. KV cache 为什么省算力? 为什么它又反过来把瓶颈推向"显存带宽"?
7. 四个 API 行为推论, 分别对应原理里的哪一站?
8. (动手) 用任一家 playground 的 tokenizer 把"客商合同额度批量导入"切分, 数一数几个 token。

04 - 进阶篇：训练三部曲与对齐

【本节问题】① 三部曲各自的数据/目标/产物？② RLHF 的完整流程？③ R1 式强化学习为什么能"练出思考"？④ 缩放定律说了什么、涌现能力是真是假？⑤ 蒸馏是什么、为什么推理模型都能出小杯？



立场声明：你不是训练工程师，但**训练方式决定模型行为**——读懂本章，你就能解释"为什么不同模型脾气不同"、"为什么思考模型不能手把手教"。

1. 第一部曲：预训练 (Pre-training)

维度	内容
数据	互联网级语料：网页/书籍/代码/论文。量级： 数万亿 token (DeepSeek V4-Pro 官方披露 33T)
目标	下一词预测 (next token prediction)，全数据一遍遍扫
算力	数千~数万卡 × 数月；成本千万~亿美元级。 占全流程 ~99% 算力
产物	基座模型 (Base) ：知识渊博的接龙机，不会对话
关键技术	缩放定律（下节）、数据清洗与配比（代码数据占比直接影响推理能力）、MoE 架构（容量↑ 单token成本↓）、Muon 等新优化器 (DeepSeek V4 披露)

缩放定律 (Scaling Laws) ——LLM 时代的摩尔定律：

- Kaplan 2020：模型越大、数据越多、算力越足，损失（预测误差）可预测地下降 → "大力出奇迹"有了理论依据
- Chinchilla 修正（2022）：**参数与数据要同比放大**——给定算力，70B 模型喂 1.4T token 优于 280B 喂 0.35T → 后来的模型都"吃得更多"
- 2024 后的第二曲线：**推理时计算缩放**（test-time compute）——o1 证明"让模型多思考"也能换智能，思考 token 是新的可缩放维度

2. 第二部曲：SFT（监督微调）

维度	内容
数据	人工/模型辅助写的 指令-回答对 ，几万~百万条，质量>>数量
目标	学会对话格式：把用户输入当 指令 执行，而非文本续写
算力	预训练的 1% 以下
产物	Instruct/Chat 模型：认识 system/user/assistant 角色

为什么工程师在乎 SFT：你写的 system prompt 之所以有"特权地位"，是因为 SFT 训练数据里 system 角色的指令被示范为"必须遵守"。**指令遵循是训出来的行为模式**（呼应 001 篇"提示是触发不是教学"）。

3. 第三部曲：对齐（Alignment）

3.1 RLHF（InstructGPT/ChatGPT 的经典配方）

- ① 人类标注员对模型的多个回答排序（哪个更有用/诚实/无害）
 - ② 用排序数据训练一个"奖励模型"（RM）——学会给回答打分
 - ③ 用 PPO 强化学习优化主模型：最大化 RM 给分，同时 KL 约束别跑太远

- **效果**：1.3B 的 InstructGPT 在人类偏好上赢了 175B 的 GPT-3——"对齐税"花得值
- **副作用**：可能变谄媚（sycophancy，迎合用户）、拒答过敏——你遇到的部分"模型脾气"来自这里

3.2 DPO（直接偏好优化）

- 简化路线：跳过奖励模型，直接从"偏好对（好答案，坏答案）"学习
- 更稳更省，成为开源模型对齐主流之一

3.3 R1 式 RL：可验证奖励强化学习（2025 范式转折）

- 场景：数学题/代码题——答案可以程序化判定对错（可验证奖励）

做法：不给标准解题步骤，只给题目和"对/错"信号，让模型自己探索
→ 答对给奖励，答错不给 → 模型自发学会"先写长思考再作答"

产物：思考链（reasoning）不是模板教出来的，是 RL 涌现的行为

- DeepSeek-R1（2025.01）公开了这条路线，触发了 2025-2026 全行业推理模型竞赛

- **对提示工程的直接影响** (001 篇呼应)：思考是内生行为 → "step by step" 指令多余甚至有害 → 控制思考用参数 (effort/thinking budget)

4. 涌现能力：真是奇迹还是幻觉？

观点	内容
涌现派 (Wei et al. 2022)	规模跨过某阈值，多步推理等能力"突然出现"
度量派 (Schaeffer et al. 2023)	部分涌现是 指标假象 ：用"完全答对才算分"的不连续指标，平滑进步也会表现为"突然跳变"；换连续指标后曲线是平滑的
工程结论	能力随规模 渐进提升 ，但"能否跨过你的任务阈值"确实是断崖的——选型时用你自己的 eval 实测，别迷信排行榜 (07 篇)

5. 蒸馏 (Distillation)：把大模型的思考装进小模型

- **一句话人话**：让小模型抄大模型的作业——用大模型生成海量"题目+优质回答 (含思考链)"，拿去 SFT 小模型
- 证据：DeepSeek-R1 发布时同步放出 1.5B~70B 蒸馏版，小杯也能推理
- 工程意义：① 端侧/本地部署小模型 (Qwen3.6-27B 单卡) 能获得远超同体积原生模型的能力；② 你的业务可以用大模型造数据、小模型降本 (06 篇成本路由的高级玩法)

6. 一张表收束：训练方式 ↔ 你看到的行为

训练阶段	决定了什么行为	对应你的工作
预训练	知识面、语言能力、代码能力	选模型的"底子" (07 篇)
SFT	对话格式、指令遵循、角色认知	system prompt 为什么有效
RLHF/DPO	语气、拒答边界、谄媚倾向	输出风格差异、过敏感拒
R1式RL	思考行为、难题攻坚	推理模型的交互方式与成本

【本节自测】

1. 三部曲的数据、目标、产物、算力占比各是什么？
2. 缩放定律和 Chinchilla 修正各说了什么？第二缩放定律"缩放"的是什么？
3. RLHF 三步流程？DPO 简化了什么？
4. R1 式 RL 为什么必须依赖"可验证奖励"？什么任务适合、什么任务不适合？
5. "涌现能力"争论的双方论点？对你的选型行为有什么实际影响？
6. 蒸馏的原理和工程价值？R1 蒸馏版说明了什么？

7. 用"训练方式↔行为"表，解释你遇到过的某一个模型"怪脾气"。

05 - 实战篇：用好推理 API（全代码）

【本节问题】① 三家 SDK 的最小调用怎么写？② 流式输出怎么消费？③ 参数怎么按任务类型配？④ 本地部署（Ollama/vLLM）什么场景值得？⑤ 调用失败的完整降级链路？

推理 API：一次调用的时序 与 一张账单的构成

上：请求从发出到流式返回的完整链路 · 下：钱花在哪、省从哪省



1. 三家最小调用（2026-09 模型名）

OpenAI（GPT-5.6 系，Responses API）

```
from openai import OpenAI
client = OpenAI() # OPENAI_API_KEY

resp = client.responses.create(
    model="gpt-5.6-terra", # Sol=旗舰 / Terra=主力 / Luna=便宜量足
    input=[
        {"role": "user", "content": "用一句话解释 KV cache 为什么能加速推理"},
    ],
    reasoning={"effort": "low"}, # 简单任务调低思考深度，省钱省时
    text={"verbosity": "low"}, # 控制回答长度
)
print(resp.output_text)
```

Anthropic (Claude)

```
import anthropic
client = anthropic.Anthropic()

resp = client.messages.create(
    model="claude-sonnet-5",
    max_tokens=1024, # 输出上限，必填
    system="你是 CTRM 系统的技术助手。回答简体中文。", # 独立参数，不要放进 messages
    messages=[
        {"role": "user", "content": "点价和升贴水是什么关系？"},
    ],
    # 注意：4.6+ 不要传 temperature/top_p（非默认值 400）
    # 注意：不要再往 messages 末尾塞 assistant 预填消息（prefill 已废弃）
)
print(resp.content[0].text)
```

DeepSeek (OpenAI 兼容格式, 国产默认选项)

```
from openai import OpenAI
client = OpenAI(api_key="sk-...", base_url="https://api.deepseek.com")

resp = client.chat.completions.create(
    model="deepseek-v4-flash", # Flash=通用快 / Pro=重推理（旧别名 deepseek-chat/reasoner 已于
    2026-07-24 退役）
    messages=[
        {"role": "system", "content": "你是贸易单证助手。"},
        {"role": "user", "content": "把这段中文品名标准化：'电解铜 500吨'"},
    ],
)
print(resp.choices[0].message.content)
```

2. 流式输出 (长回答的体验救星)

```
# Anthropic 流式示例 (SSE)
with client.messages.stream(
    model="claude-sonnet-5", max_tokens=1024,
    system="...", messages=[{"role": "user", "content": "写一份点价流程说明"}],
) as stream:
    for text in stream.text_stream: # 每个 token/片段实时到达
        print(text, end="", flush=True)
```

- 原理呼应 03 篇：生成就是逐 token 的，流式只是把每个 token **边生成边推给你** (SSE)
- 指标：**TTFT (首字延迟)** 主要受 prefill (prompt 长度) 影响；整体时间 $\approx$ TTFT + 输出 token 数 / 吞吐
- Web 应用 (你的 Vue 前端) 用 `fetch` + `ReadableStream` 消费 SSE 即可

3. 参数配置速查（按任务类型）

任务	模型档位	温度/思考	关键参数
分类/抽取/结构化	Luna / Flash / Haiku 级	temp 0 / effort low	response_format=JSON Schema
日常写作/总结	Terra / Sonnet / Flash	temp 0.7 或默认	verbosity 按需
复杂推理/审查	Sol / Pro / Opus 级	effort high（默认）	控制思考预算上限
Agent 主控	Sonnet/Terra 级	effort medium	并行工具调用开启
海量批处理	任意	—	batch 端口（全场半价）

原则（呼应 001 篇）：**能用参数解决的别写进提示词；提示词里只放业务语义。**

4. 结构化输出（跨厂商通用模式）

```
from openai import OpenAI
client = OpenAI()

resp = client.chat.completions.create(
    model="deepseek-v4-flash",
    messages=[{"role": "user", "content": "从邮件抽取点价要素: <mail>...</mail>"}],
    response_format={
        "type": "json_schema",
        "json_schema": {
            "name": "pricing_request",
            "schema": {
                "type": "object",
                "properties": {
                    "items": {"type": "array", "items": {"$ref": "#/$defs/item"}},
                },
                "$defs": {
                    "item": {
                        "type": "object",
                        "properties": {
                            "commodity": {"type": ["string", "null"]},
                            "quantity": {"type": ["number", "null"]},
                            "delivery_month": {"type": ["string", "null"]},
                        },
                        "required": ["commodity", "quantity", "delivery_month"],
                    }
                }
            }
        }
    ]
)
```

- 约束解码：引擎层直接禁掉不合 schema 的 token → 结构正确率 ~100%（05 篇 001 专题修复循环的更强上位替代）

- Anthropic 用 structured outputs / `output_config.format` ; 不支持的模型退回"提示词要 JSON + 校验重试"

5. 本地部署：Ollama / vLLM（数据主权场景）

工具	定位	适用
Ollama	一键拉起，桌面级	开发体验、隐私草稿、demo。 <code>ollama run qwen3.6:27b</code>
vLLM	生产级推理服务（PagedAttention、continuous batching）	团队内网服务、高并发。吞吐比 naive 方案高一个量级

Ollama: 5 分钟上手（Qwen3.6-27B 单卡 24GB 可跑，中文场景首选本地模型）
`ollama run qwen3.6:27b`

vLLM: 生产部署
`vllm serve Qwen/Qwen3.6-27B --max-model-len 32768 --gpu-memory-utilization 0.9`

什么时候值得自部署（决策清单）：

- [] 数据不能出内网（合规/涉密）→ 必须本地
- [] 调用量巨大且任务简单（分类/抽取）→ 小模型本地，边际成本趋近电费
- [] 需要微调私有领域模型（蒸馏/LoRA）→ 本地闭环
- [] 只是"不想付 API 费"但调用量小 → 别，自部署的运维成本会教做人

6. 失败与降级链路（生产必写）

调用失败 → 判断错误类型：
└ 429 限流 → 指数退避重试（读 `Retry-After` 头）
└ 5xx/超时 → 换同档位备选模型重试一次
└ 上下文超限 → 触发历史裁剪/摘要压缩（008 专题）后重试
└ 结构化输出校验失败 → 带错误信息重试（001 篇修复循环）
└ 仍然失败 → 降级到规则兜底/排队人工，并打告警

配套：所有调用记录 模型版本+提示词版本+token用量 （06 篇监控的地基）。

【本节自测】

1. 三家 SDK 的 system 写法差异？Anthropic 两个"不要传"的参数是什么、为什么？
2. 流式输出的原理（用 03 篇语言解释）？TTFT 由什么决定？
3. 写出五类任务的参数配置（不看表）。
4. 约束解码和"提示词要 JSON+重试"的强度差异？
5. Ollama 和 vLLM 的定位差别？自部署的四条判断清单？

6. 手写降级链路伪代码：429、超限、校验失败三条分支。
7. (动手) 用 Ollama 跑通 qwen3.6:27b (或更小杯) , 对比同任务与 DeepSeek API 的质量/延迟。

06 - 生产篇：成本、延迟、质量三角

【本节问题】① 一张 LLM 账单由几部分构成、哪部分最肥？② 缓存/batch/路由各自能省多少？③ 延迟优化的抓手？④ 模型退役怎么办（真实案例）？⑤ 怎么防止"月底账单吓一跳"？

推理 API：一次调用的时序 与 一张账单的构成

上：请求从发出到流式返回的完整链路 · 下：钱花在哪、省从哪省



1. 账单解剖学 (2026-09 实价)

计费项	说明	量级关系 (以主流价为参照)
输入 token	prompt 全部内容	基准价 (\$0.2~\$10 /1M)
输出 token	回答 + 思考 token	输入的 4~6 倍——最肥的一块
缓存命中输入	相同前缀复用 KV	输入价的 0.1~0.2 倍
长上下文附加费	GPT-5.6 > 272K ×2; Gemini > 200K 翻倍; Claude 无附加	容易被忽略的暗坑
batch 异步	全场约 5 折	非实时任务的默认选择

成本公式：

单次成本 = 输入token × 输入单价 × (1-缓存折扣×命中率) + 输出token(含思考) × 输出单价 × (1-batch折扣)

Agent 场景警报：多轮工具循环中，历史每轮重新计输入费——一个 50 轮的 Agent 会话，**重复前缀的浪费可达总成本的 60%+**。三个对策：稳定前缀缓存化、历史裁剪（008 专题）、`previous_response_id` 类会话复用（OpenAI Responses API）。

2. 省钱抓手（按 ROI 排序，可直接执行）

抓手	做法	预期收益
① 模型分级路由	简单任务（分类/抽取/改写）→ Luna/Flash/Haiku 级；疑难 → 旗舰	综合 -60%~-90%
② Prompt Caching	稳定内容（system+术语表+few-shot）固定在 prompt 开头；可变内容放后	重复调用场景 -50%~-80% 输入成本
③ 输出瘦身	verbosity=low / 格式约束 / "只输出 JSON 不要解释"	输出是输入 5 倍价，砍输出=砍大头
④ 思考预算控制	简单任务 effort=low / 关闭 thinking（Qwen enable_thinking=false）	推理模型场景常常 -50%+
⑤ batch 化	非实时任务走 batch 端口	-50%
⑥ 历史裁剪	多轮会话滚动摘要、工具结果不全文回流	Agent 场景 -40%+
⑦ 本地小模型兜底	高频简单任务本地化（Qwen3.6-27B / Mistral Small 4）	边际成本≈电费

反模式警告：为省钱把旗舰任务塞给小模型 → 返工率上升 → "每次调用便宜了 80%，总成本反升 30%"。❌
径永远是：每次成功任务的成本，不是每次调用的成本。

3. 延迟优化（体验工程）

手段	原理（03 篇呼应）
流式输出	把总延迟切成首字延迟，体感大幅改善
缩短 prompt	prefill 时间 ∝ 输入长度（TTFT 的大头）
降低思考深度	effort=low：思考 token 少，首字和总时长双降
并行调用	无依赖的多路请求同时发（注意限流）
就近部署	国内业务选 DeepSeek/Qwen/阿里云百炼，跨境延迟差可观
投机解码	本地 vLLM 场景的小模型草稿+大模型验证（了解即可）

预算参考（Sonnet 5 级）：TTFT 1~3s（1k prompt） / 5~15s（100k prompt）；吞吐 50~100 token/s。
超预期先查：prompt 是否过长、是否误用高 effort。

4. 质量守护：不要只看"感觉变好了"

- **固定评测集**：50~200 条真实业务样本 + 判定规则（呼应 001 篇 eval），每次换模型/换参数全量跑
- **A/B 灰度**：新旧配置各 5% 流量，比"成功任务成本+人工复核率"两个指标
- **监控面板最小集**：调用量、token 用量（输入/输出/思考/缓存命中四分开）、P95 延迟、错误率、每千次调用成本
- **漂移告警**：成本或复核率周环比 >20% 即排查（常见：厂商静默调价/升级、上游数据格式变化）

5. 模型退役：真实案例与迁移清单

2026-06-15: **Claude Sonnet 4 / Opus 4 全线退役，调用直接报错，无自动 failover**。这不是孤例——OpenAI/Google 同样常态化下架旧模型。

迁移 Checklist:

- 订阅厂商 deprecation 公告（邮件/更新日志）
- 代码里模型名集中管理（配置/网关），全仓搜硬编码
- 读迁移指南：**prefill** 废弃、采样参数禁用、**thinking** 配置变更、JSON 转义差异
- 全量 eval 跑新旧对比（看分布不只看均值）
- 灰度 5% → 观察 badcase → 全量
- 更新成本基线与监控阈值

（对应 001 篇 06 章的 prefill 迁移映射表——那次退役波及大量线上 pipeline。）

6. 一页成本优化 SOP（贴在团队 wiki）

新功能上线前：

1. 估算：预估日均调用 × 单次 token → 月成本区间（悲观=3×）
2. 选型：默认 Terra/Sonnet/Flash 级；证明需要再升旗舰
3. 结构：稳定前缀在前（吃缓存）；可变内容在后
4. 输出：格式约束 + verbosity 下限
5. 非实时：全部走 batch

上线后：

6. 监控四分账单（输入/输出/思考/缓存）
7. 每月复核一次：能否降档、能否增缓存命中、能否裁历史

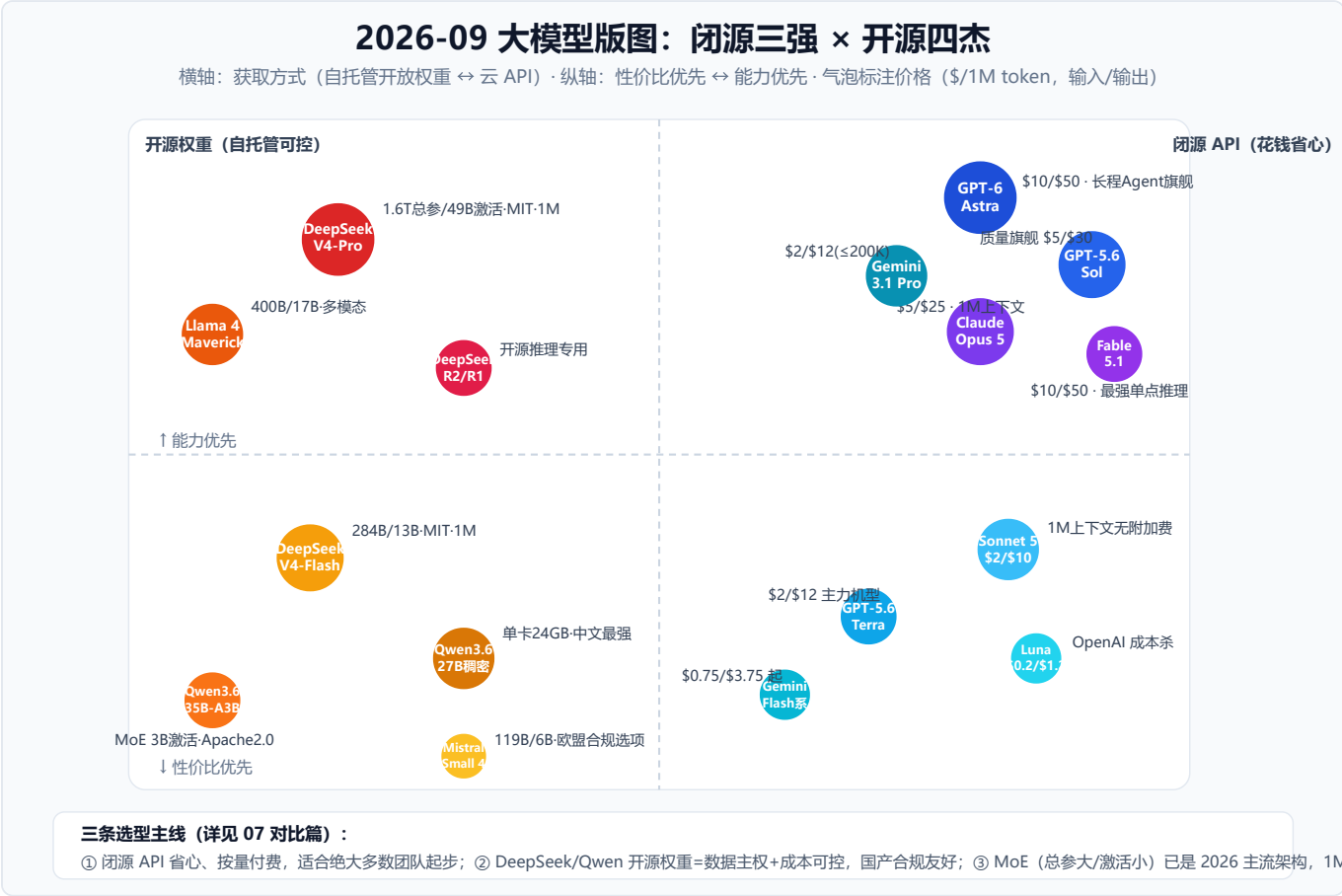
【本节自测】

1. 账单五项各自的量级关系？哪项最肥？
2. Agent 多轮场景为什么成本爆炸？三个对策分别作用在哪个环节？
3. "每次调用便宜 80% 但总成本升 30%"的反模式是怎么发生的？正确口径是什么？
4. 缓存命中的条件？你的提示词结构要怎么排布？
5. TTFT 和吞吐分别由什么决定？两条优化路径？
6. 模型退役案例给我们的工程教训是什么？（至少三条）

7. 手算：日均 1 万次调用，prompt 2k token (90% 缓存命中) + 输出 500 token, Sonnet 5 (\$2/\$10, 缓存 0.1×) 月成本多少？改走 batch 再算一遍。

07 - 对比篇：2026 模型格局全景与选型

【本节问题】① 闭源三强和开源四杰各家的牌面？② MoE 是怎么统一 2026 的？③ 你的场景该选谁？④ 价格之外的隐藏差异？



时效声明：以下为 2026-09 快照（多家比价站交叉核对），价格/阵容每季度都变——结构比数字长寿，学结构。

1. 闭源三强（API 优先视角）

	OpenAI	Anthropic	Google
旗舰	GPT-5.6 Sol（\$5/\$30） / GPT-6 Astra（\$10/\$50，长程 Agent）	Claude Opus 5（\$5/\$25） / Fable 5.1（\$10/\$50，最强单点推理）	Gemini 3.1 Pro（\$2/\$12 ≤200K，超则翻倍）
主力	GPT-5.6 Terra（\$2/\$12）	Claude Sonnet 5（\$2/\$10，1M 无附加费）	Gemini 3.5/3.8 Flash（\$0.75~1.5）
便宜量足	GPT-5.6 Luna（ \$0.20/\$1.20 ）	Haiku 4.5（\$1/\$5）	Flash-Lite（\$0.10/\$0.40 全场最低价）

	OpenAI	Anthropic	Google
上下文	400K~1M (>272K 输入 ×2)	1M (无附加费)	1M (>200K 翻倍)
独门	reasoning_effort/verbosity 参数化、Responses API 推理上下文复用	合同/代码长文一致性口碑、effort 自适应思考、缓存 97.5% off (Fable 5.1)	多模态全家桶、Flash 性价比、超长上下文
风格	参数化控制，工程化程度高	企业级稳定性叙事、长任务可靠性	多模态与价格侵略性

2. 开源四杰（自托管/数据主权视角）

模型	架构	上下文	许可证	一句话定位
DeepSeek V4-Pro	MoE 1.6T/49B 激活	1M	MIT	开源天花板，代码/推理最强（预训练 33T token，压缩稀疏注意力）
DeepSeek V4-Flash	MoE 284B/13B	1M	MIT	性价比主力，API 极便宜
Qwen3.6-27B	稠密（Gated DeltaNet 混合注意力+MTP）	262K（YaRN 可扩展 1M）	Apache 2.0	单卡 24GB 可跑，中文最强本地模型
Qwen3.6-35B-A3B	MoE 35B/3B	262K→1M	Apache 2.0	3B 激活的效率怪物
Llama 4 Scout/Maverick	MoE 109B/17B、400B/17B	Scout 理论 10M / Maverick 1M	Llama 社区许可（<700M MAU 商用）	多模态开源标杆、西方生态默认底座
Mistral Small 4	MoE 119B/6B	—	Apache 2.0	欧盟合规选项，单 A100 可跑
DeepSeek R1 系（蒸馏 1.5B~70B）	稠密/蒸馏	—	MIT	开源推理小杯

3. MoE 是怎么统一 2026 的（结构理解）

稠密模型：每个 token 过全部参数 → 容量想要大，单 token 成本就爆炸
MoE： 路由器为每个 token 从 N 个专家里挑 2~8 个 → 总参管"知识库存量"，
激活参管"单 token 成本" → 两者解耦，第一次让 1.6T 模型可以便宜地跑

- 读名口诀：DeepSeek-V4-Pro 1.6T/49B、Qwen3.6-35B-A3B（A=激活）、Maverick 400B/17B——**第二个数才是推理成本的决定量**
- 你的显存规划看总参（量化后），你的延迟/吞吐预期看激活参

4. 选型决策树（贴给团队用）

数据能出内网吗？

└ 不能 → 本地部署：中文场景 Qwen3.6-27B（单卡）；多模态 Llama 4 Scout；推理小杯 R1-Distill-32B；生产并发 vLLM 托管

└ 能 → 调用量级？

└ 小/中（<100 万次/月）→ API：

└ 追质量：GPT-5.6 Sol / Claude Opus 5 / Gemini 3.1 Pro

└ 平衡（默认）：Claude Sonnet 5 / GPT-5.6 Terra

└ 海量简单任务：GPT-5.6 Luna / Gemini Flash-Lite

└ 大（>100 万次/月）→ 混合：高频简单任务 → DeepSeek/Qwen API 或本地小模型
低频疑难 → 旗舰 API；中间层 Sonnet/Terra

中国业务加分项：DeepSeek/Qwen/百炼 的合规、备案与延迟优势

5. 价格之外的隐藏差异（容易被忽略）

维度	差异点
缓存策略	各家命中折扣与最小前缀长度不同（Claude Fable 5.1 缓存 97.5% off；有的 1024 token 起缓存）
长上下文附加费	GPT/Gemini 有阈值翻倍档；Claude 无—— 长文档场景选 Claude 可能更便宜
思考计费	各家对思考 token 的计费与暴露方式不同（完整可见/仅摘要）
工具调用质量	同样"支持工具"，并行调用可靠度差异大——Agent 场景必实测（004 专题）
中文 token 效率	DeepSeek/Qwen 词表对中文更友好，同内容 token 数常少 20-30%
合规	国产模型备案齐全；出海数据出境评估、欧盟 AI Act → 影响 API vs 自托管选择

6. 本笔记的立场（可检验的判断）

1. **默认档**：Sonnet 5 / Terra / Flash 是 2026 年的"生产默认档"——90% 的业务任务不需要旗舰

2. **国产组合拳**：DeepSeek（质量）+ Qwen（本地/多模态）+ 百炼/DeepSeek API（合规通道）已能覆盖国内企业大部分场景

3. **1M 上下文是营销数字优先**：先问你的场景是否真有 >100K 的单次输入；没有，就别为它付溢价（context rot 也在等你）

4. **结构判断**：MoE + 长上下文 + 可调思考 = 2026 三大件；选型时核对这三项的参数化程度，比看跑分有用

【本节自测】

1. 三强各自的"旗舰/主力/便宜"三档型号与价格？

2. 开源四杰各自的定位一句话？中文本地场景选谁、为什么？

- 36 / 40 -

3. MoE 解耦了什么和什么？"第二个数才是成本决定量"指的是什么？
4. 走一遍决策树：你的 CTRM 系统（数据敏感、日均 2 万次抽取+少量复杂审查）怎么选？
5. 隐藏差异六条中，哪三条最可能改变你的账单？
6. 本笔记"1M 上下文是营销数字优先"的判断，你同意吗？给出你场景的论据。

08 - 自测篇：题库与面试题（主动回忆 + 间隔重复）

用法：合上笔记作答，再核对；错题入错题本，按 D1/D3/D7/D21 重做。

L1 概念题 (01-02 篇)

1. 语言模型在建模什么？用一个公式 + 一句人话。
2. token 是什么？中文与英文的 token 换算比例？为什么"按字数估成本"必错？
3. embedding 把什么变成了什么？RAG 检索用的 embedding 和模型内部的 embedding 是一回事吗？
4. 注意力机制里 Q/K/V 各自的角色？ n^2 代价指什么？
5. decoder-only + 自回归是什么意思？流式输出为什么是"天然"的？
6. 上下文窗口的定义？"输入+输出共享窗口"是什么意思？
7. MoE 的总参/激活参分别决定什么？读一下 Qwen3.6-35B-A3B 这个名字。
8. KV cache 解决什么问题？它和 prompt caching 的区别？
9. temperature/top_p 各控制什么？temperature=0 保证什么不保证什么？
10. 思考 token 是什么、怎么计费？哪些模型有、旋钮叫什么？

L2 原理与训练题 (03-04 篇)

1. 画出六站前向流水线，标注 Transformer 块内的两个子层。
2. 位置编码 (RoPE) 解决什么问题？没有它模型会犯什么错？
3. Prefill 和 Decode 两阶段各自在干什么、瓶颈各是什么？TTFT 由哪个决定？
4. KV cache 为什么省算力？为什么瓶颈又转向显存带宽？
5. 缩放定律与 Chinchilla 修正各说了什么？第二缩放定律缩放的是什么？
6. RLHF 三步流程？DPO 省掉了什么？R1 式 RL 的"可验证奖励"为什么关键？
7. "涌现能力"争论双方论点？对你的选型行为有什么影响？
8. 蒸馏的原理？R1 蒸馏版对你本地部署的意义？

L3 应用与生产题 (05-06 篇)

1. 三家 SDK 的 system 写法差异？Anthropic 的两个"不要传"参数及原因？
2. 为以下任务配置模型档位+参数：① 邮件分类 ② 合同风险审查 ③ 海量历史单据归档 ④ Agent 主控。
3. 写出"约束解码 vs 提示词要 JSON+重试"的强度对比和各自适用条件。
4. 自部署决策清单四条，逐条判断你公司 CTRM 场景（数据敏感、日均 2 万次抽取）该不该本地化。
5. 手算成本：日均 1 万次、prompt 2k token（90% 缓存命中）、输出 500 token，Sonnet 5（\$2/\$10、缓存 0.1×）月成本？走 batch（5 折）呢？

- 6. Agent 多轮场景成本爆炸的机理与三个对策？
- 7. 模型退役迁移 checklist 七条，默写至少五条。
- 8. 降级链路：为"限流 / 上下文超限 / 结构化校验失败"各设计一条处理路径。

L4 面试题（大厂 AI 应用岗风格）

- 1. 向完全不懂 AI 的 CTO 解释：为什么 LLM API 按 token 计费、为什么输出比输入贵 5 倍？（考原理变 现）
- 2. 为什么 2026 年主流全是 MoE？稠密模型还有生存空间吗（举 Qwen3.6-27B 反例）？（考架构理解）
- 3. 长上下文窗口从 8K 涨到 1M，"lost in the middle"问题解决了 吗？工程上怎么对抗？（考 context rot，联动 008）
- 4. 设计一个"成本可控的文档问答系统"：模型分级、缓存设计、批处理、监控指标全链路。（考 05+06 综 合）
- 5. RLHF 和 R1 式 RL 的本质区别是什么？各自适合什么任务？（考训练范式）
- 6. 你如何评估"该用推理模型还是通用模型"？给出可量化的决策规则。（考成本质量权衡）
- 7. 为什么 Claude 4.6+ 禁用了手动采样参数？这反映了模型行业什么趋势？（自适应取代手动控制）
- 8. 国产模型 vs 闭源三强，从 token 效率、合规、价格、生态四个维度做结构化对比。（考格局认知）

📅 间隔重复计划

时点	动作
D1	全部 L1 + 标记错题
D3	L1 错题 + L2 全部
D7	L2 错题 + L3 全部（含成本手算）
D21	L3 错题 + L4 任选 3 题；费曼终极检验：写 500 字《给 Java 同事的 LLM 原理速览》
M2	重做错题本；用当月最新价格表重算 L3-5

错题本模板

错题 #N 日期：____ 来源：L_ - __

- 我的答案：
- 正确答案/出处：<篇章.节>
- 错因：概念不清 / 原理没懂 / 没算过 / 记混了
- 重做：D3 / D7 / D21

本专题一手资料索引

资料	用途
各厂商模型页与定价页（2026-09 快照，注明季度复核）	07 篇数据来源
DeepSeek V4 技术报告（V4 系列架构披露：CSA/HCA、Muon、33T token）	04/07 篇
Qwen3.6 技术报告（Gated DeltaNet、MTP、YaRN）	04/07 篇
Anthropic 《Effective context engineering for AI agents》(2025-09)	context rot 来源
Chinchilla / Kaplan 缩放定律论文	04 篇
DeepSeek-R1 论文（arXiv:2501.12948）	R1 式 RL
Wei et al. 涌现论文 vs Schaeffer et al. 度量论文	04 篇争论
Transformer 原论文《Attention Is All You Need》(arXiv:1706.03762)	03 篇地基